SimSpect: Automated litmus testing for Gem5 Spectre defenses

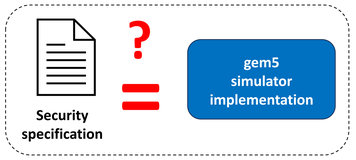
Stanford | ENGINEERING
Electrical Engineering

Anna Eaton, Nicholas Mosier, Caroline Trippel
Stanford University

Introduction

SimSpect is a framework for validating that simulated Spectre defenses uphold their advertised security requirements by testing them with **security litmus tests** automatically generated from **formal defense specifications.**

- **Spectre defenses** vary according to what architectural state they protect from speculatively leaking and how/when they protect it, thereby requiring different security litmus tests to stress and expose bugs in their implementations. These
- SimSpect introduces:
  1. A **formal specification language** for specifying programmer-transparent, comprehensive Spectre defenses
  2. A **pipeline** for synthesizing security litmus tests from formal defense specifications, running the tests on simulator implementations of the defense, and examining the result to detect bugs



Formal language

Grammar

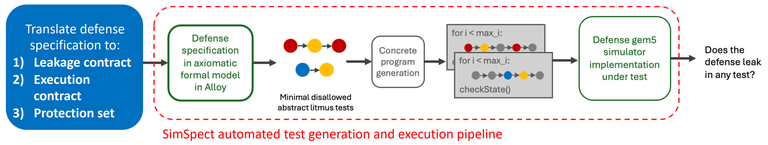
A defense is captured in SimSpect's formal language by specifying three definitions that make up its secure speculation scheme:

1. **Leakage contract** — the set of pairs of microarchitectural transmitters and their unsafe operands considered by the defense (e.g. implicit v.s. explicit transmitters)
2. **Execution contract** — the characterization of what control flow and data flow is allowed on the microarchitecture, particularly specifying which type of instruction is considered speculative (e.g. non-resolved ctrl instructions or all non-resolved instructions)
3. **Protection set** — the set of architectural state that should not leak speculatively (e.g. all memory v.s. specific addresses)

4. Protection mechanism — implied enforcer of the behavior of the preceding three definitions: Typically mandates that the protection set is not leaked through all the transmitters in the leakage contract, under speculation as defined in the execution contract

	STT	SPT	Recon
Leakage contract	(load, address) (ctrl, conditions) ***stores are not transmitters**	All implicit and explicit transmitters	All implicit and explicit transmitters
Execution contract	All control and data speculation	All control and data speculation	All control and data speculation
Protection set	All memory	Speculatively accessed data and non-speculative secrets	Speculatively accessed data and non-speculative secrets, leaked by pointers

Pipeline



SimSpect automated test generation and execution pipeline

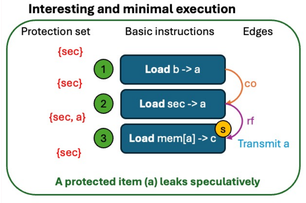
1. **Template generation**
   - Given formal representation of secure speculation scheme, capture the minimal disallowed litmus tests in an abstract form
   - Use a base axiomatic hardware model in **Alloy Analyzer** model to generate all possible such tests
   a. **Relaxations**
      - We want to first reduce to minimal-state representations of each leaky test to reduce execution overhead
   b. **Protection set exercise**
2. **Enumeration**
   - Generate real .asm snippets, annotated with branch predictor or address predictor state, from an abstract test trace
   - Fill in how speculation arises, and which X86 instructions fit in the abstract cases
   - Add complications that legally should not change speculation behavior including timing, extra dependencies, etc.
3. **Execution**
   - Given some functional .asm snippets, annotated with branch predictor or address predictor state, prime and run in gem5
   - Use new branch and address predictors that explicitly set which predictions are made
   - Execute the tests repeatedly, and use explicit measurement of leakage

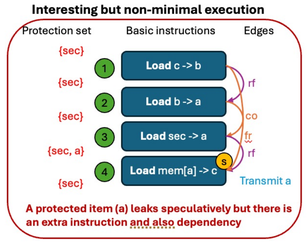
Minimal and interesting executions

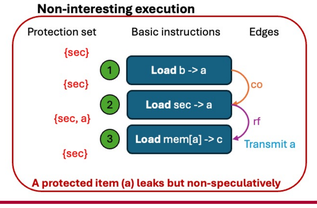
We relax our violating executions until they reach a threshold where any future relaxation would cause them to violate.

Bug 1 – Recon STT

Recon stalls transmitters until oldest load that the sensitive operand depends on, should be the youngest.







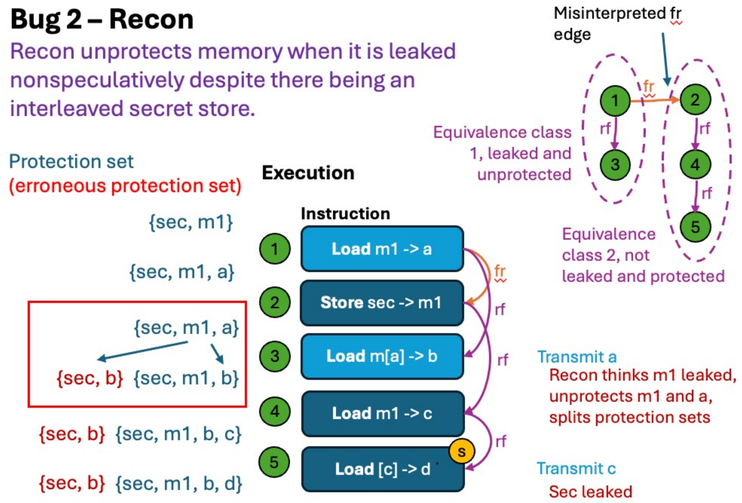
Legal relaxations:
A selected execution must violate the secure speculation scheme, and under any relaxation not violate the secure speculation scheme
- **RS**: remove arch. state element
- **RI**: remove instruction
- **RB:** remove Boolean
- **RA, RD, RM, RR**: remove operand edges between state and instruction (affects rf, co, and fr edges)

Legend:
Instruction nodes: Load, store, div, ctrl, other
Booleans flags: executed, handled_pred, faulted, retired
State nodes: memory, registers
Edge types: po (implied in shown examples), rf, fr, co (memory consistency model terms generalized to register and memory operations)

Protection set tracking

Bug 2 – Recon

Recon unprotects memory when it is leaked nonspeculatively despite there being an interleaved secret store.

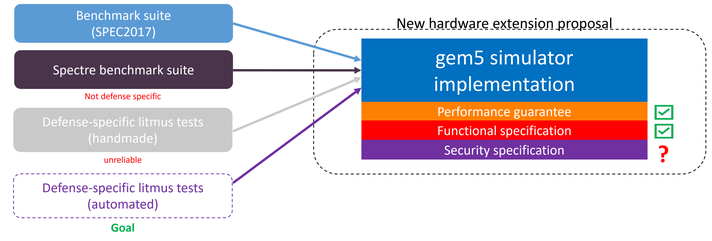


Hypothesis: Consider the equivalence classes of arch. state tracked by a defense. Poor protection set tracking occurs when non-rf edges are misinterpreted to collapse equivalence classes
Equivalence set to the output of instruction i at its execution time = what state could be unprotected by transmission at i = i.(^(~rf+rf) & ^(~po))

Related work

Related work

- **AMuLeT** verifies speculative non-interference, a particular defense property, but is not flexible for defense-specific security contracts



Ongoing work

- We modeled STT, and were able to capture at least one known bug
- We would like to generate buggy implementations in a principled way to allow for more comprehensive testing